

Operating System (ICE 227)

Md. Azizul Hakim
Dept. of ICE, DIU.



Process Concept

- An operating system executes a variety of programs:
 - Batch system –**jobs**
 - Time-shared systems –**user programs or tasks**
- **Process**—a program in execution; process execution must progress in sequential fashion

Process Concept

- Program is **passive** entity stored on disk (**executable file**), process is **active**
- Program becomes process when executable file loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc
- One program can be several processes
- Consider multiple users executing the same program

Process in a memory

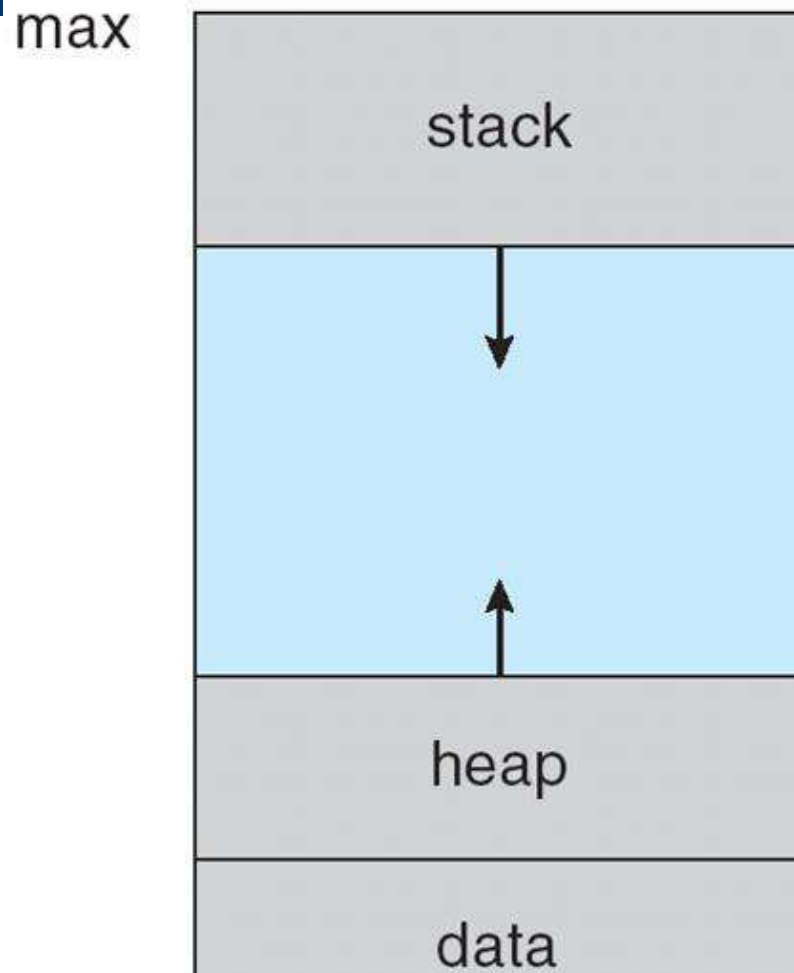
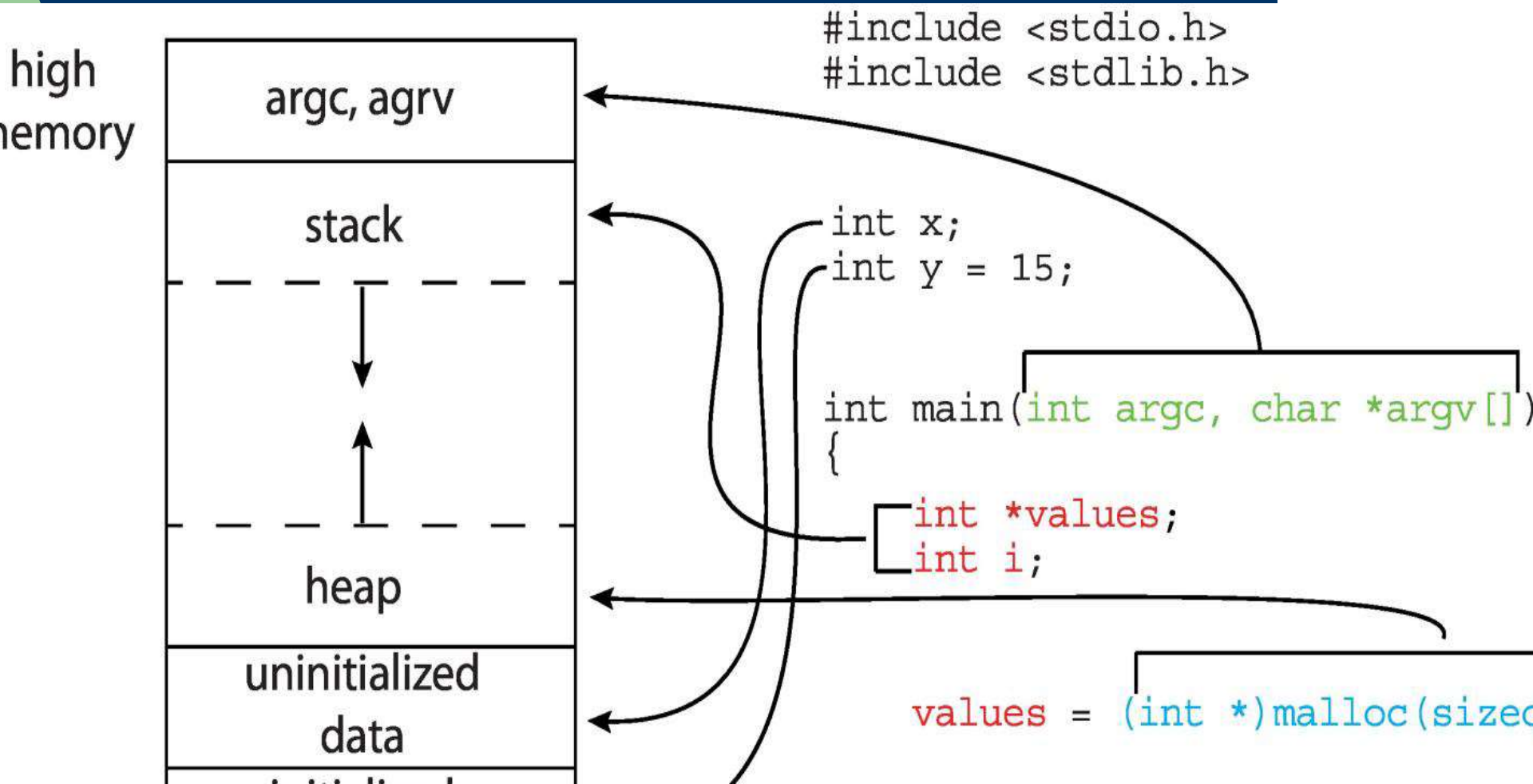


Diagram
shows
process in
a memory

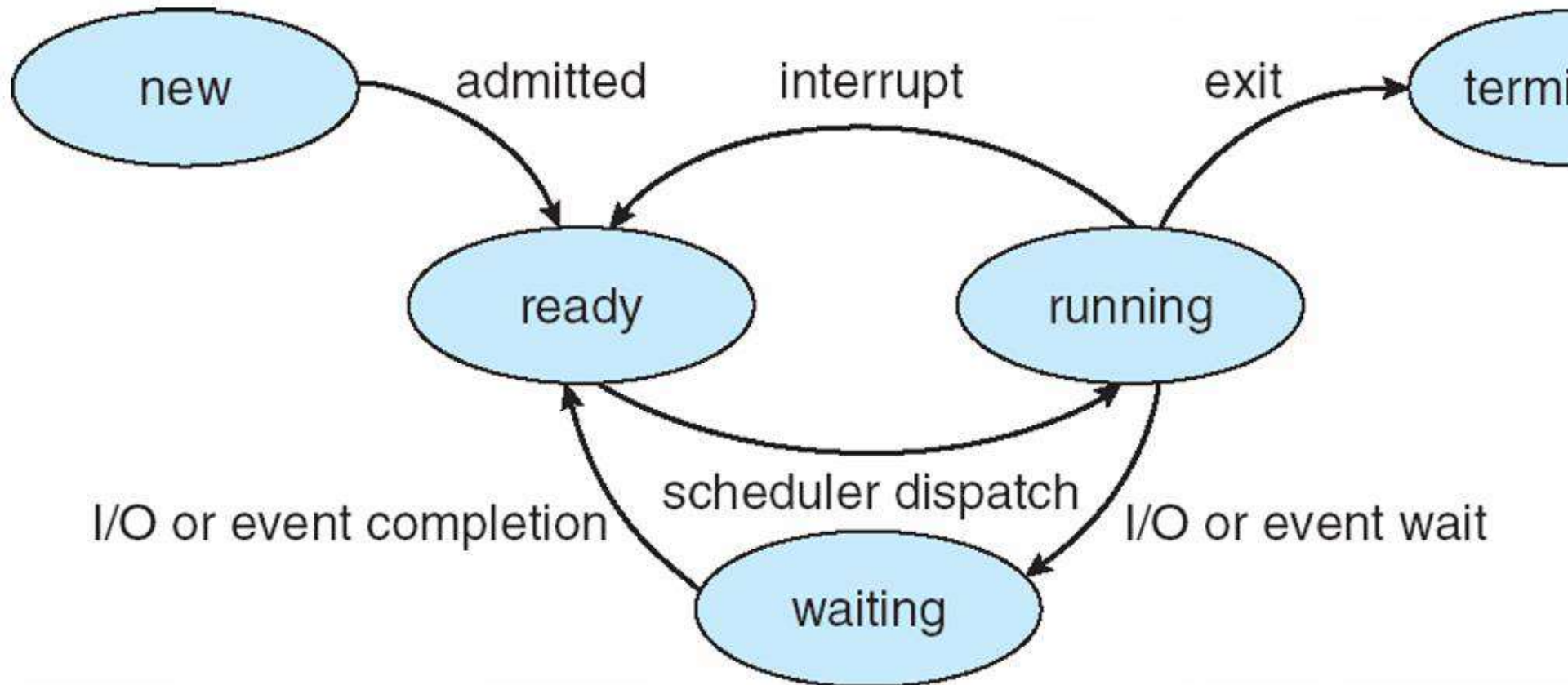
Memory layout of a c program



Process State

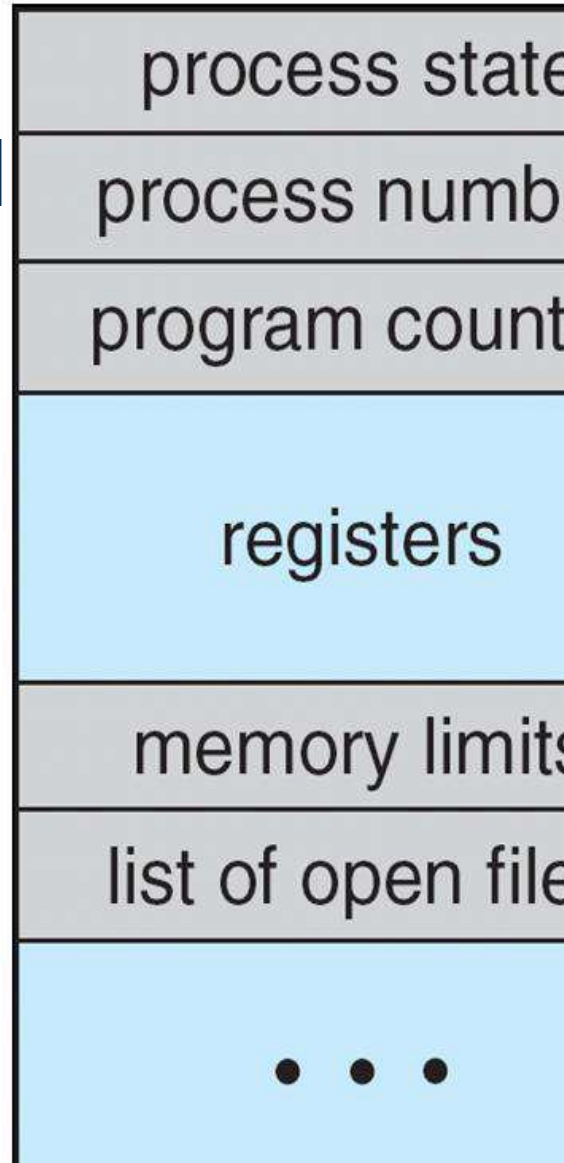
- As a process executes, it changes **state**
- **new**: The process is being created
- **running**: Instructions are being executed
- **waiting**: The process is waiting for some event to occur
- **ready**: The process is waiting to be assigned to a processor
- **terminated**: The process has finished execution

Diagram of a process state



Process Control block

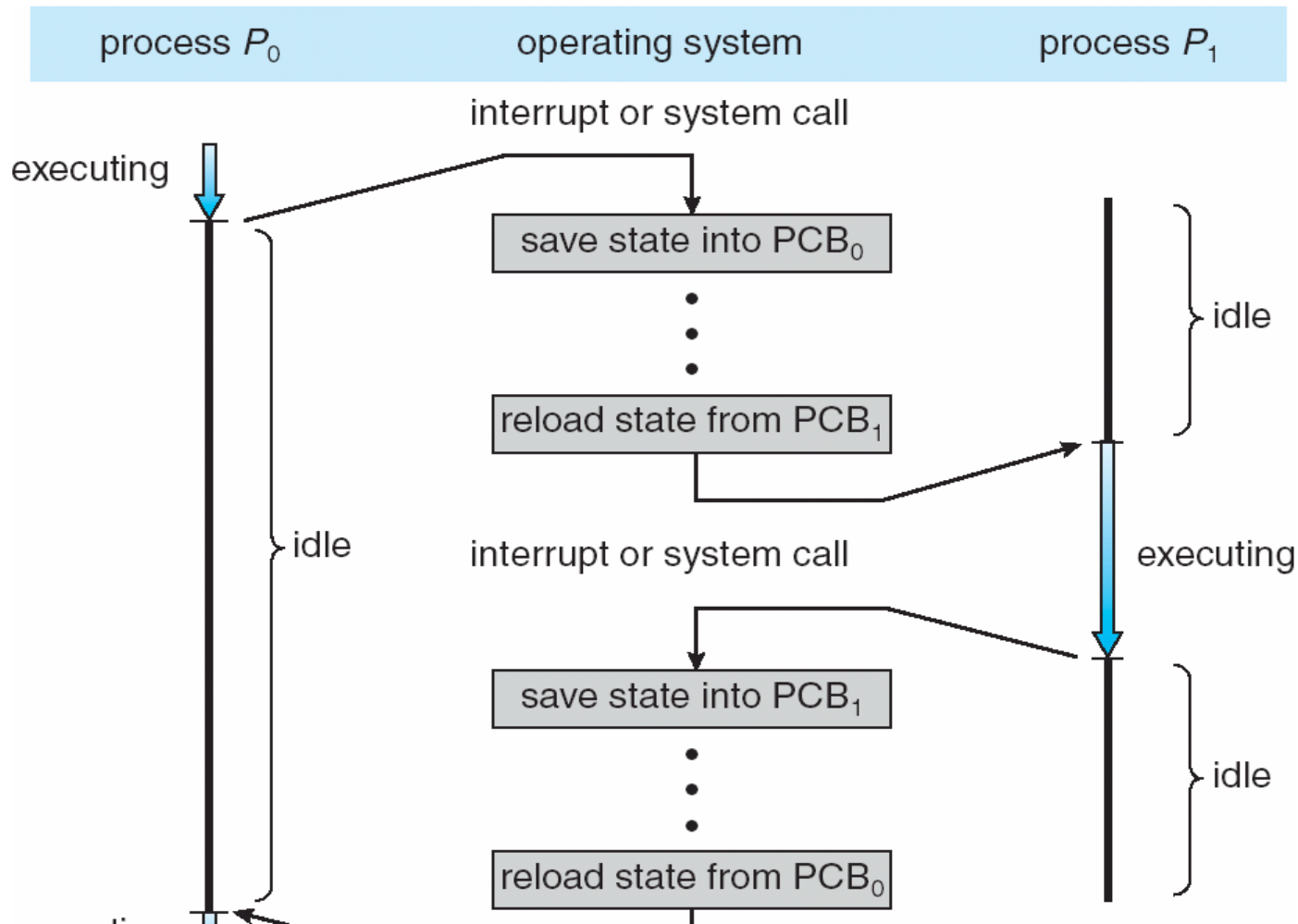
- Process state –running, waiting, etc
- Program counter –location of instruction to next execute
- CPU registers –contents of all process-centric registers
- CPU scheduling information-priorities, scheduling queue pointers
- Memory-management information –memory allocated to the process
- Accounting information –CPU used, clock time elapsed since start, time limits
- I/O status information –I/O devices allocated to process, list of open files



Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the PCB
- Context-switch time is overhead; the system does no useful work while switching
- –The more complex the OS and the PCB  the longer the context switch
- •Time dependent on hardware support
- –Some hardware provides multiple sets of registers per CPU  multiple contexts loaded at once

CPU Switch from process to process

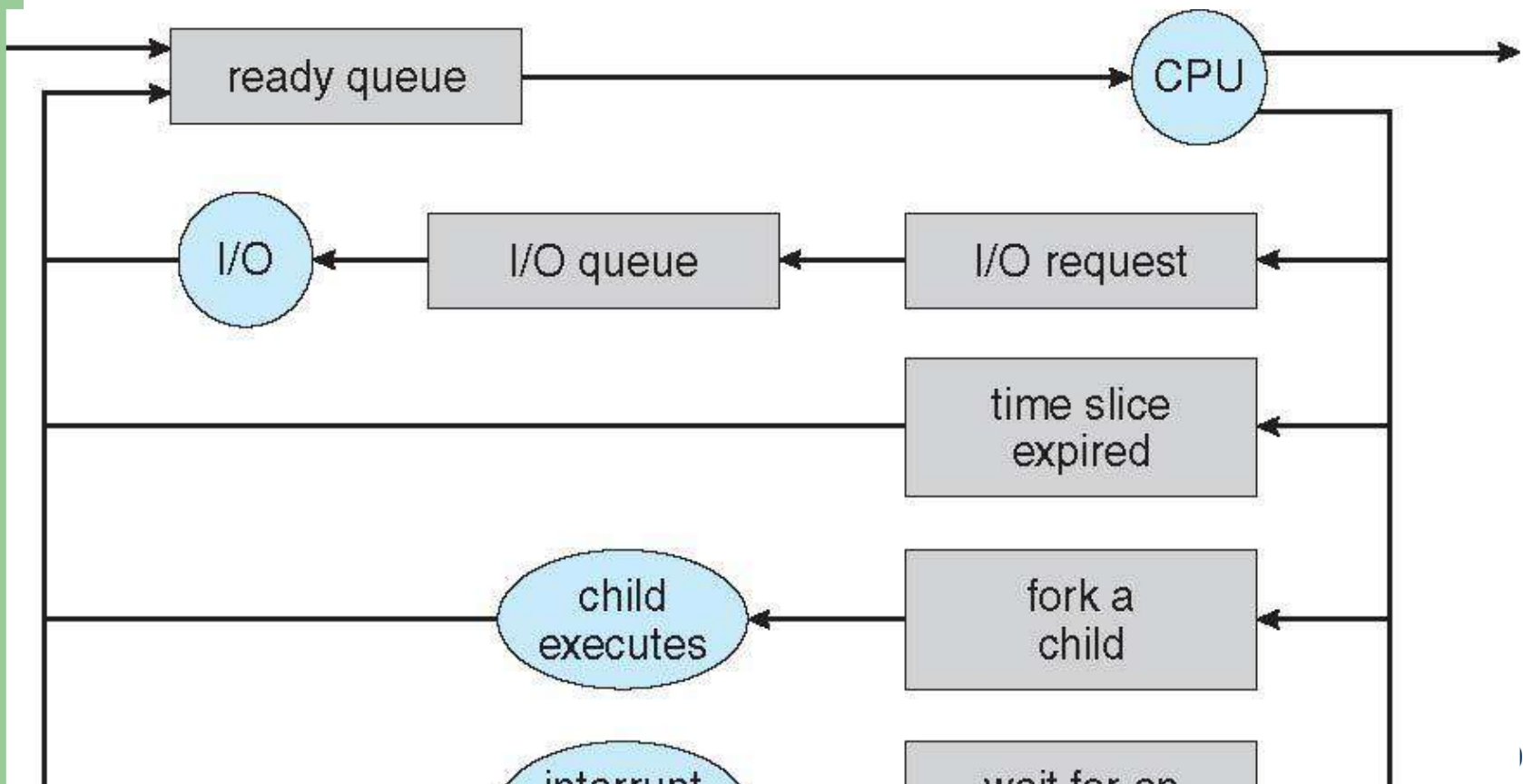


Process Scheduling

- Maximize CPU use, quickly switch processes onto CPU for time sharing
- **Process scheduler** selects among available processes for next execution on CPU
- Maintains **scheduling queues** of processes
- **Job queue** –set of all processes in the system
- **Ready queue** –set of all processes residing in main memory, ready and waiting to execute
- **Device queues** –set of processes waiting for an I/O device
- Processes migrate among the various queues

Representation of Process Scheduling

- Queueing diagram represents queues, resources, flows



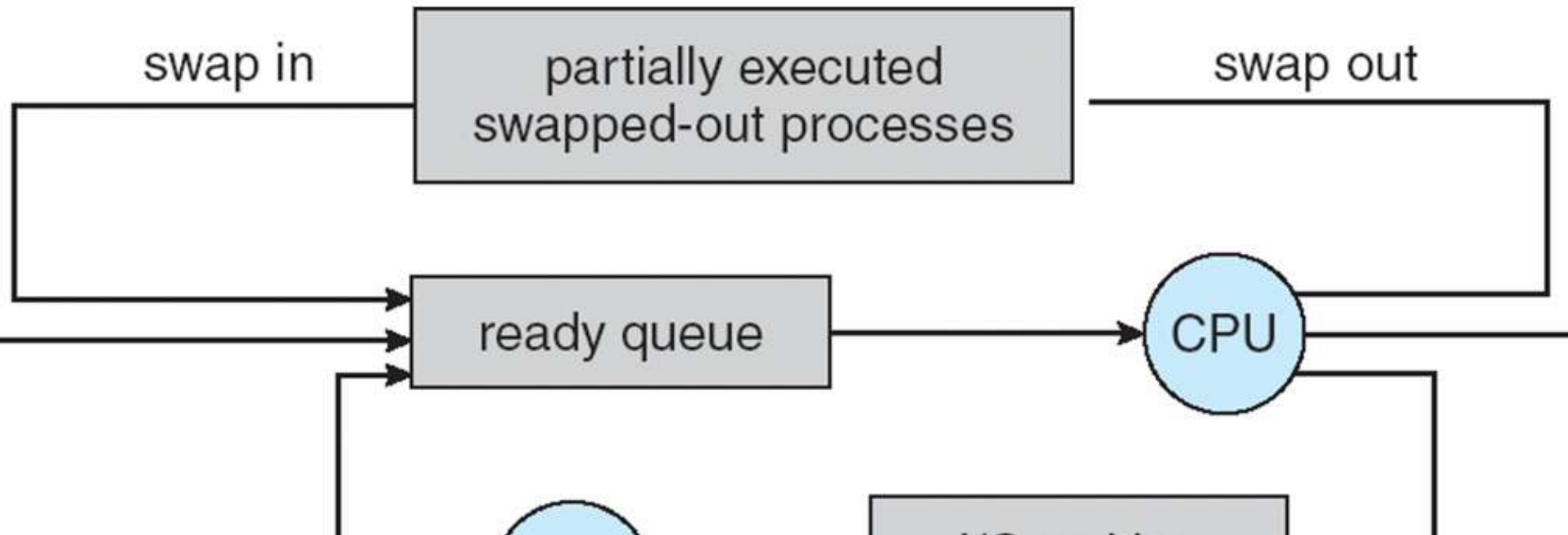
Scheduler

- **Short-term scheduler** (or **CPU scheduler**) –selects which process should be executed next and allocates CPU
- Sometimes the only scheduler in a system
- Short-term scheduler is invoked frequently (milliseconds) □(must be fast)
- **Long-term scheduler** (or **job scheduler**) –selects which processes should be brought into the ready queue
- Long-term scheduler is invoked infrequently (seconds, minutes) □(may be slow)
- The long-term scheduler controls the **degree of multiprogramming (number of process in main memory)**
- Processes can be described as either:
- **I/O-bound process**—spends more time doing I/O than computations, many short CPU bursts

Medium Term Scheduling

Medium-term scheduler can be added if **degree of multiple programming needs to decrease** (reduce the number of processes from the main memory)

Remove process from memory, store on disk, bring back in from disk to continue execution: **swapping**



Multitasking in Mobile System

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
- iOS
- Single **foreground** process-controlled via user interface (**older versions**)
- Multiple **background** processes—in memory, running, but not on the display, and with limits
- Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback
- Multiple foreground application through **split screen** and **few limits in the newer versions**.
- Android runs foreground and background, with fewer limits
- Background process uses a **service** to perform tasks
- Service can keep running even if background process is suspended

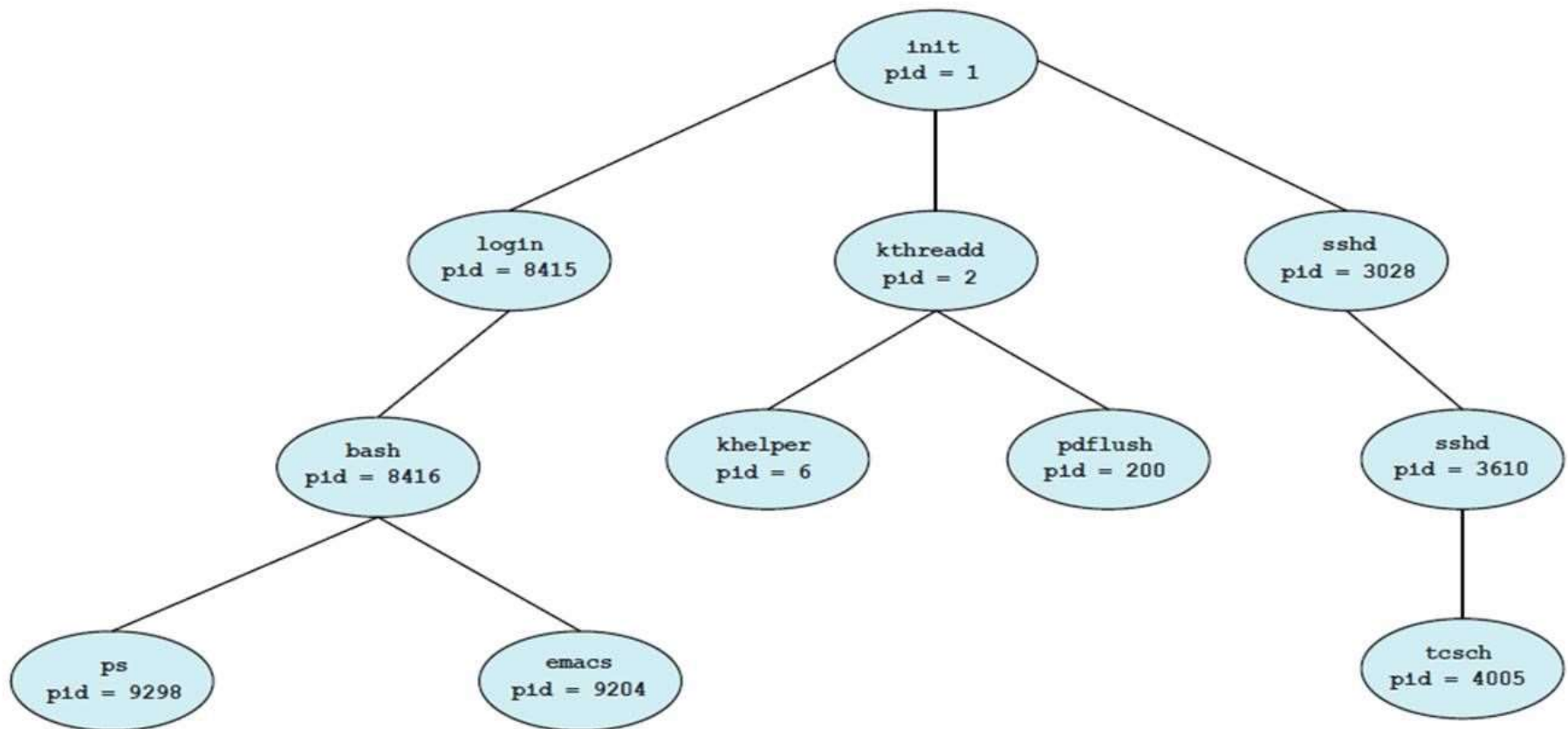
Operations on Processes

- System must provide mechanisms for:
 - process creation,
 - process termination,
 - and soon as detailed next

Process Creation

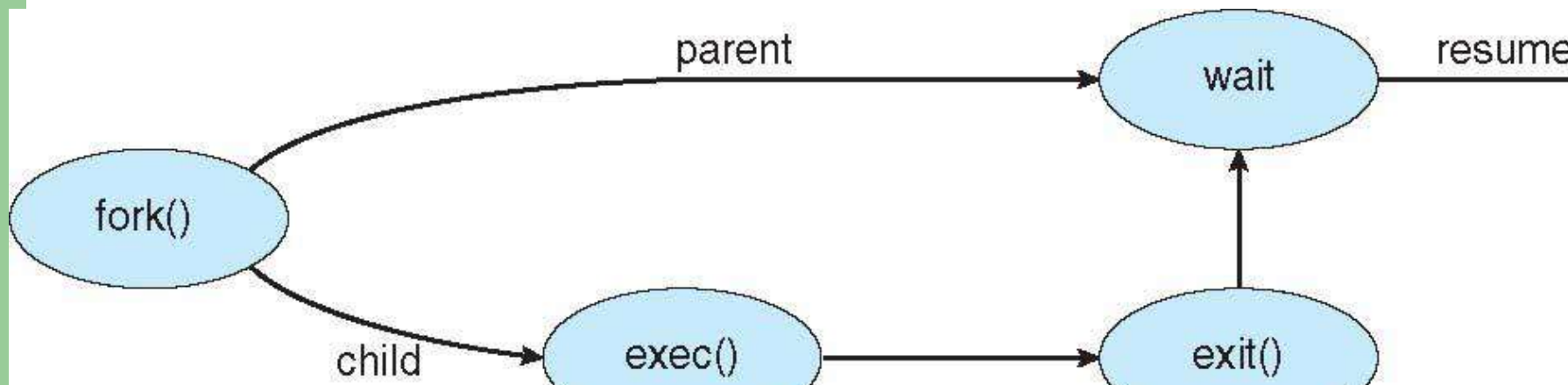
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes
- Generally, process identified and managed via a **process identifier (pid)**
- Resource sharing options
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- Execution options
 - Parent and children execute concurrently
 - Parent waits until children terminate

A Tree of Processes in Linux



Process Creation

- Address space
 - Child duplicate of parent
 - Child has a program loaded into it
- UNIX examples
 - fork()** system call creates new process
 - exec()** system call used after a **fork()** to replace the process' memory space with a new program



C program Forking separate Process

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

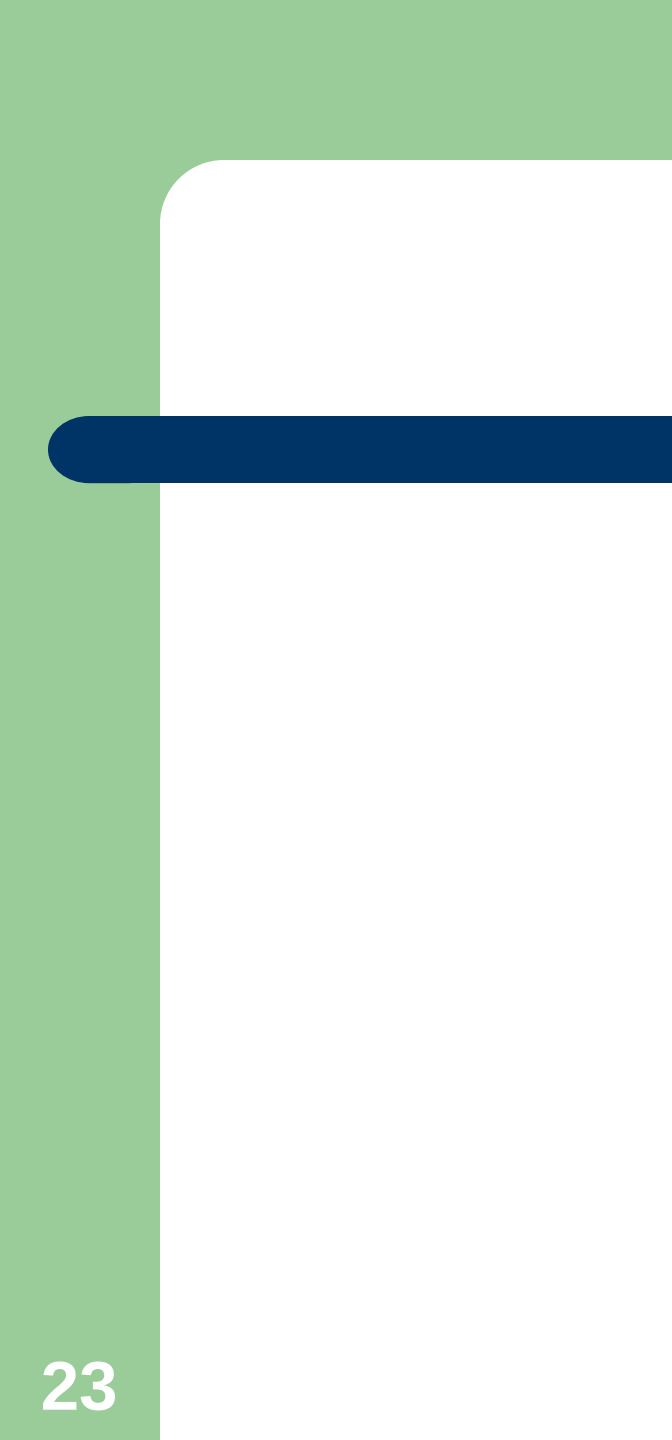
    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
}
```

Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system
- Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting, and the operating system does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exist if its parent has terminated. If a process terminates, then all its children must also be terminated.
 - cascading termination.** All children, grandchildren, etc. are terminated.
 - The termination is initiated by the operating system.
- The parent process may wait for termination of a child process by using the **wait()** system call. The call returns status information and the pid of the terminated process
 - pid= wait(&status);**
- A **zombie** process or defunct process is a process that has completed execution but still has an entry in the process table as its parent process didn't invoke a wait() system call.
- An **orphan** process is a computer process whose parent process has finished or terminated (without wait() call), though it (child process) remains running itself (Linux and Unix assign the init() process as the new parent).



Thank You